

JavaServer Pages

Objetos Implícitos · Aula 3

De request a response: como uma página recebe dados, responde — e decide se troca de URL ou não. Hoje: out, request, response, redirect e dispatcher.



Prof. Domingos Bernardo

Instituto Federal de São Paulo — Curso Técnico

As Peças de uma Página JSP



Na aula passada vimos as cinco sintaxes que fazem uma página JSP funcionar: Scriptlet, Declaração, Expressão, Comentário e Import.



Scriptlet

Lógica Java: `<% %>`



Declaração

Variáveis: `<%! %>`



Expressão

Exibir valor: `<%= %>`



Import

Trazer classes: `<%@ %>`

Vocês já sabem colocar lógica Java na página. Hoje: como uma página conversa com a outra.

Nesta Aula



1. Objetos Implícitos

Visão geral: os 9 objetos



2. out

Escrevendo na página



3. request

Recebendo dados do cliente



4. response

Controlando a resposta



5. redirect

`response.sendRedirect()`



6. dispatcher

`RequestDispatcher.forward()`



7. Redirect vs. Forward

O comparativo



8. Prática, exemplos e exercícios

Mão na massa

Nove Objetos, Zero Instanciação

Todos disponíveis em qualquer página JSP, sem precisar declarar ou instanciar nada.

out	Escreve diretamente no HTML de resposta	Hoje
request	Dados da requisição: parâmetros e atributos	Hoje
response	Controla a resposta: headers, status e redirect	Hoje
session	Espaço de dados de um usuário entre requisições	Próxima aula
application	Contexto global, compartilhado por todos os usuários	Em breve
config	Parâmetros de inicialização do servlet	Em breve
exception	Captura erros em páginas de erro	Em breve
pageContext	Acesso centralizado a todos os outros objetos	Em breve
page	Referência à própria instância da página (this)	Em breve

out

`out.print()`

```
<%  
    String turma = "LP2 - Noturno";  
    out.print("<h2>Turma: " + turma + "</h2>");  
  
    for (int i = 1; i <= 3; i++) {  
        out.println("<p>Aluno " + i + "</p>");  
    }  
%>
```

O que é

out é o objeto implícito que escreve diretamente no HTML de resposta — funciona como um `System.out.println()`, mas o destino é a página que o navegador vai exibir.

Quando usar

- Montar HTML dentro de um laço (for, while)
- Misturar texto fixo com valores calculados
- Enviar conteúdo linha a linha, sob condição

listaPares.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<body>
  <h1>Números Pares de 1 a 20</h1>
  <ul>
    <%
      for (int n = 1; n <= 20; n++) {
        if (n % 2 == 0) {
          out.println("<li>" + n + "</li>");
        }
      }
    %>
  </ul>
</body>
</html>
```

O laço roda inteiramente no servidor — o navegador só recebe a lista de já prontos.

request

`request.getParameter()`

```
<%  
    String prontuario = request.getParameter("prontuario");  
    String senha = request.getParameter("senha");  
    String metodo = request.getMethod();  
%>
```

O que é

request representa a requisição HTTP enviada pelo navegador — é dele que tiramos parâmetros de formulário, dados da URL e cabeçalhos enviados pelo cliente.

Quando usar

- Ler campos de um `<form method="get">` ou `"post"`
- Validar dados enviados pelo usuário
- Guardar informação para outra página usar (`setAttribute`)

login.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<!DOCTYPE html>
<html>
<body>
  <form action="processaLogin.jsp" method="post">
    <table>
      <tr><td>Prontuário:</td>
        <td><input type="text" name="prontuario"/></td></tr>
      <tr><td>Senha:</td>
        <td><input type="password" name="senha"/></td></tr>
      <tr><td colspan="2">
        <input type="submit" value="Acessar"/></td></tr>
    </table>
  </form>
</body>
</html>
```

O formulário envia prontuario e senha — do outro lado, quem recebe os dois valores é o objeto request.

response

`response.setContentType()`

```
<%  
    response.setContentType("text/html; charset=UTF-8");  
    response.setHeader("Cache-Control", "no-cache");  
    response.setStatus(200);  
%>
```

O que é

response representa a resposta HTTP que o servidor vai devolver — controla tipo de conteúdo, cabeçalhos e código de status. Como veremos a seguir, também pode mandar o navegador para outra URL.

Quando usar

- Definir o tipo de conteúdo (HTML, JSON, PDF...)
- Configurar cabeçalhos HTTP (cache, cookies)
- Sinalizar erros com códigos de status (404, 500...)

statusAluno.jsp

```
<%@ page contentType="application/json; charset=UTF-8" %>
<%
    String id = request.getParameter("id");

    if (id == null) {
        response.setStatus(400);
        out.print("{\"erro\": \"parâmetro id é obrigatório\"}");
    } else {
        response.setStatus(200);
        out.print("{\"id\": \"" + id + "\", \"status\": \"ok\"}");
    }
%>
```

Trocamos o content type para application/json — o response decide não só O QUE volta, mas COMO o navegador deve interpretar a resposta.

Redirect

`response.sendRedirect()`

```
<%
    if (autenticado) {
        response.sendRedirect("boasVindas.jsp");
    } else {
        response.sendRedirect("login.jsp?erro=1");
    }
%>
```

O que é

`sendRedirect` envia ao navegador um código 302 com um cabeçalho `Location` — é uma instrução para o PRÓPRIO NAVEGADOR fazer uma nova requisição para outra URL.

Quando usar

- Depois de um POST, para evitar reenvio do formulário (padrão Post/Redirect/Get)
- Levar o usuário para fora da aplicação atual
- Quando a URL final deve aparecer para o usuário

processaLogin.jsp — com redirect

```
<%  
    String prontuario = request.getParameter("prontuario");  
    String senha = request.getParameter("senha");  
  
    if ("12345".equals(prontuario) && "12345".equals(senha)) {  
        response.sendRedirect("boasVindas.jsp");  
    } else {  
        response.sendRedirect(  
            request.getContextPath() + "/login.jsp?erro=1");  
    }  
%>
```

Repare: depois do sendRedirect, a URL na barra do navegador muda para boasVindas.jsp — e um SEGUNDO request é criado do zero.

Dispatcher (Forward)

RequestDispatcher

```
<%  
    request.setAttribute("usuario", prontuario);  
  
    RequestDispatcher rd =  
        request.getRequestDispatcher("boasVindas.jsp");  
    rd.forward(request, response);  
%>
```

O que é

`RequestDispatcher.forward()` encaminha a MESMA requisição para outro recurso dentro do servidor — tudo em uma única requisição HTTP, sem o navegador perceber.

Quando usar

- Reaproveitar dados já carregados no request (`setAttribute` / `getAttribute`)
- Montar uma página a partir de includes (cabeçalho, corpo, rodapé)
- Quando a URL não deve mudar para quem está navegando

processaLogin.jsp — com forward

```
<%  
    String prontuario = request.getParameter("prontuario");  
    String senha = request.getParameter("senha");  
  
    if ("12345".equals(prontuario) && "12345".equals(senha)) {  
        request.setAttribute("usuario", prontuario);  
  
        RequestDispatcher rd =  
            request.getRequestDispatcher("boasVindas.jsp");  
        rd.forward(request, response);  
    } else {  
        response.sendRedirect("login.jsp?erro=1");  
    }  
%>
```

Agora a URL continua sendo processaLogin.jsp — mas o conteúdo exibido já é o da boasVindas.jsp.

Redirect vs. Forward

Critério	<code>redirect (response)</code>	<code>forward (dispatcher)</code>
Nº de requisições HTTP	2 — nova requisição	1 — mesma requisição
URL no navegador	Muda	Continua a mesma
Dados do request (<code>setAttribute</code>)	Perdidos	Preservados
Pode sair da aplicação atual?	Sim, qualquer URL	Não, só dentro do servidor
Desempenho	Uma viagem extra ao servidor	Mais rápido — sem ida e volta
Como lembrar	"Vá você mesmo" — o navegador decide	"Eu entrego por você" — o servidor decide

Cadastro de Aluno

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%
    String nome = request.getParameter("nome");
    String curso = request.getParameter("curso");

    request.setAttribute("nomeAluno", nome);
    request.setAttribute("cursoAluno", curso);

    RequestDispatcher rd =
        request.getRequestDispatcher("confirmacao.jsp");
    rd.forward(request, response);
%>
```

salvarAluno.jsp recebe os dados do formulário, guarda no request e encaminha para confirmacao.jsp — sem o navegador fazer uma nova requisição.

Cadastro de Aluno — Confirmação

```
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<body>
  <h1>Cadastro confirmado!</h1>
  <p>
    Aluno: <%= request.getAttribute("nomeAluno") %><br/>
    Curso: <%= request.getAttribute("cursoAluno") %>
  </p>
</body>
</html>
```

A URL na barra continua /salvarAluno.jsp — mesmo mostrando o conteúdo de confirmacao.jsp. Os dados vieram do MESMO request, com `getAttribute`.

Enquete da Turma

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%
    String opcao = request.getParameter("opcao");

    // aqui entraria a lógica para salvar o voto
    // (banco de dados, arquivo, etc.)

    response.sendRedirect("resultado.jsp?opcaoVotada=" + opcao);
%>
```

salvarVoto.jsp processa o voto e redireciona para resultado.jsp — repare que o dado vai na URL, não no request.

Enquete da Turma — Resultado

```
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<body>
  <h1>Obrigado por votar!</h1>
  <p>
    Você votou em: <%= request.getParameter("opcaoVotada") %>
  </p>
</body>
</html>
```

Usamos `request.getParameter`, não `getAttribute` — depois do redirect, o request antigo já não existe mais. Só sobrevive o que veio na URL (ou na session).

Volta ao Projeto de Vocês

1

**Criem
processaLogin.jsp**

Leiam prontuário e
senha com
`request.getParameter`

2

Guardem um dado

Usem
`request.setAttribute`
antes do forward

3

**Encaminhem com
forward**

`RequestDispatcher`
para a página de
boas-vindas

4

**Troquem por um
redirect**

Troquem o forward
por um `sendRedirect`
e comparem

5

**Anotem a
diferença**

O que mudou na
URL? E nos dados do
request?

Não precisa ficar bonito — o objetivo é sentir a diferença entre forward e redirect na prática.

Para Ir Além



Oracle — JavaServer Pages Technology

oracle.com/java/technologies/jspt.html



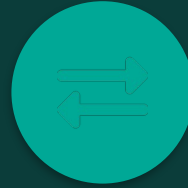
Oracle Docs — The Java EE 5 Tutorial (Request Dispatch)

docs.oracle.com/javaee/5/tutorial/doc/bnagy.html



TutorialsPoint — JSP Tutorial

tutorialspoint.com/jsp/index.htm



Obrigado pela Atenção!

out, request, response, redirect e dispatcher — como uma página conversa com a outra, com ou sem trocar de URL.



Próxima aula: session e application — os objetos implícitos que atravessam várias requisições.

Prof. Domingos Bernardo

SPOLBP2 · Lista de exercícios disponível na pasta Exercícios